agens-3: An end-to-end autonomous network simulator for ns-3

Kemal Yağız Daşkıran, Ali Berat Ak
Department of Electronics and Communications Engineering
Istanbul Technical University
Istanbul, Turkey
Email: {daskiran21, akb22}@itu.edu.tr

***Abstract*—Inspired by the emergence of AI agents aiming to assist procedures across various branches of engineering and science, we propose agens-3, an end-to-end, large language model-based agent framework for ns-3 network simulations. We expand the previous works by reducing the need for a human-in-the-loop mechanism by implementing self-correcting state-of-the-art autonomous orchestration methodologies, and retrieval-based lookup tools for the gathering of factually correct and up-to-date information regarding network simulation standards. We experimented our model with various standard-compliant network environments to assess the agent's capability in configuring topologies and analyzing traffic flows. Our experiments provide successful results across the three test-cases, and provide insights related to how large language models' think, plan and act to solve challenges related to network simulations that can be used in further research related to the automation of network simulations in ns-3.**

***Index Terms*—AI Agents, Network Simulations, Large Language Models, ns-3**

I. Introduction

Following the recent approaches towards large language model-based agents, there are advancements in automating tasks related to coding, writing and management. However, considering the complexity of network simulations and limitations on available training data related to ns-3, large language models present as unreliable for generating compiling ns-3 simulation codes without any further guidance or human supervision [1]. To address these limitations, we propose agens-3, an end-to-end multi-agent network simulation framework for ns-3 network simulator, inspired by the work "Toward Generative 6G Simulation: An Experimental Multi-Agent LLM" [1]. Furthermore, we implemented modern approaches to our framework, such as retrieval augmented generation (RAG) [2], multi-agent collaboration and tools for running and analyzing network simulations with ns-3. We aimed to gather generalizable results across ns-3 simulation scenarios to create an autonomous "Human-language-to-network-simulation" pipeline.

Our work differs from Rezazadeh et al., as it relied heavily on human intervention and fixes, our work covers complete autonomy, evading any manual instructions and human-in-the-loop approaches [1] by implementing self-correcting measures by implementing an multi-agent system with different specialists as AI agents to plan, execute and evaluate the simulation process. Furthermore, we are expanding the work by implementing retrieval pipelines with ns-3 manuals and code examples to increase code generation performance with Retrieval Augmented Generation [2]. Our modular and low-level orchestration approach enabled us to gather the underlying thought processes of specialist agents' to grasp a better understanding of the process. Our work connects the bridge of state-of-the-art Agentic AI solutions and network simulations. To validate the capabilities of agens-3, we conducted extensive experiments across three representative network simulation scenarios with varying degrees of complexity. We evaluated the framework's performance based on code compilation rates, logical correctness, and the alignment of simulation outputs with expected network behaviors.

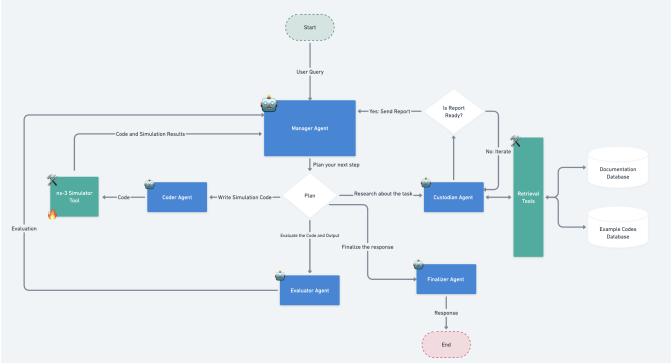


Fig. 1: Overview of the agens-3 architecture

II. Architecture

The architecture of agens-3 consists of five different specialized large language model-based agents interacting with each other and four different tools. Tools are used for retrieving information across the database and simulating the produced code through the ns-3 environment to create generalizable results. The overview of the agens-3 framework can be seen in Fig. 1.

Overall, agens-3 expects the user to provide a fully specified ns-3 network simulation scenario as a prompt. The prompt can explicitly describe the network topology, node roles, mobility models, protocol configuration, and simulation objectives. Afterwards, The user query is transferred to the Manager Agent to plan and make the decision for the next step. Manager

Agent's decision escalates to the required agent to complete their task, and the results from the specified specialist agent are returned to the manager for the next decision. This process concludes and agens-3 provides a final response when the Manager Agent decides if the knowledge gathered is sufficient enough for satisfying the objectives provided by the initial query. This process, from start to finish, is fully autonomous and does not require human intervention.

A. Knowledge Retrieval

Even though large language models can store factual information, the strict API requirements of ns-3 and the evolving state of the syntax may lead to frequent errors in creating ns-3 codes. Due to the fact that ns-3 components rely heavily on facts to create stable and sound code, factually correct information becomes a necessity for our architecture. Even with the modern reasoning capabilities, models perform better if a curated and factually correct information is directly provided [3].

Therefore, we proposed retrieval augmented generation-based approaches by implementing semantic retrieval tools with vector databases containing of ns-3 documentation and example simulation codes for the *Custodian Agent* to utilize. In more detail, our vector space is constructed from two primary document sources: the ns-3 Model Library documentation for Release ns-3.46, which provides up-to-date information on available simulation models, and the ns-3 Manual for Release ns-3.46, which describes the simulator architecture, core abstractions, and recommended usage patterns for the architecture to follow.

III. Components

A. Tools

Our agens-3 architecture utilizes certain tools across agents to gain certain insights autonomously throughout the process.

1) ns-3 Simulator Tool: Used by Coder Agent to automatically test and measure results from the generated ns-3 code, both in C++ and Python coding languages. The tool runs the tests on the local computer and returns precise results. Compared to its original, this tool is further optimized for more precise detection of used coding language and the simulation performance.

2) C++ & Python Compiler Tool: Used for running, compiling and debugging code in Python and C++ coding languages. We observe that agens-3 tends to utilize this tool for tasks that are not related to ns-3 but require coding nonetheless, such as precise calculation.

3) Web Search Tool: Used for gathering any missing information related to the task from the internet. However, we observe that this tool may lead to hallucinations and context dilution due to the fact that searches might give out information that is unrelated or not up-to-date.

4) Search Database Tool: Utilizes Retrieval Augmented Generation to return relevant chunks from the vector space of curated documents and example codes for ns-3 simulations related to the query. Used by the Custodian Agent for gathering relevant pieces of information.

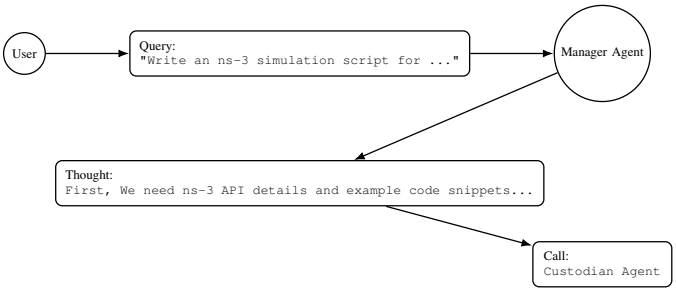


Fig. 2: Example Manager Agent flow with visible internal scheme.

B. Agents

The agens-3 framework consists of four different specialized large language model-based agents for differing tasks, such as planning, data gathering, coding and interpretation. To create a simulation scenario that satisfies the user's requirements, the agents communicate with eachother for multiple iterations.

1) Manager Agent: The manager agent's role is to supervise and plan the process, replacing human-in-the-middle, in the agens-3 architecture. During inference, the manager agent is provided with the user's initial request, system prompt for guidance and the previous actions taken by the agens-3.

Afterwards, the manager agent is asked to plan the next step to continue with the process. The manager's response contains which agent to continue with, and the detailed instructions for the following agent to follow.

Overall, the Manager Agent can call the following agents:

- **Custodian Agent**: for gathering information and research.
- **Coder Agent**: for writing and simulating ns-3 code.
- **Evaluator Agent**: for evaluating the ns-3 simulation code and results.
- **Finalizer Agent**: for generating the final response.

2) Custodian Agent: The Custodian Agent acts as the main researcher for the agens-3 architecture that generates reports. With the request from the Manager Agent, Custodian Agent can utilize retrieval augmented generation with tools to search through the vector space and gather factually stable information regarding the request. Custodian Agent returns a final report, explaining the concepts Manager Agent asked to address in detail, along with code examples and implementation methods.

For each request, the Custodian Agent iterates up to 6 times before preparing the final report, reading through the documentation at each iteration. However, it can also conclude early and create a report early.

3) Coder Agent: Taking the user's request, the guidance from the Manager Agent, and the created report from the Custodian Agent into consideration, Coder Agent generates network simulation codes for ns-3. Afterwards, Coder Agent uses the ns-3 tool to simulate the proposed network, generates

outputs and provides the information to the Manager Agent to consider the next step.

4) Evaluator Agent: Rather than traditional evaluation methods or relying on simple metrics such as the compilability of the generated code, LLM-based agentic networks excel when evaluated and recieve feedback like humans [4]. Rich evaluative feedback at the full thought and action trajectory paves a clear path for the framework to follow in the following steps. For total autonomy in the agens-3 framework, we proposed an autonomous feedback agent for this purpose, the Evaluator Agent.

When called, Evaluator Agent evaluates the written code for network simulations and the outputs. Evaluator Agent offers fixes, improvements or comments regarding the quality, suitability and the optimization of the code if necessary.

IV. Experimental Setup

A. Evaluation Metrics

1) Iterative Refinement Count: This metric quantifies the autonomous debugging capability of the **GPT-5 mini** backbone by measuring the number of refinement cycles required to synthesize syntactically correct and compilable source code.

2) KPI Deviation: The accuracy of the simulations is assessed by comparing the agent's results against human-coded "Ground Truth" scripts. This deviation shows how closely the agent's output matches expert standards.

3) Cyclomatic Complexity: This metric measures the logical depth of the generated code by analyzing the number of decision points (loops, if-conditions etc.) and independent paths.

4) Resource Efficiency: Computational overhead is analyzed by monitoring run-time resource utilization, specifically focusing on volatile memory (RAM) allocation and CPU execution time. This metric provides insight into the efficiency of the underlying engineering decisions made by the agent.

B. Test Scenarios

To ensure a comprehensive validation across different layers of the protocol stack, three distinct simulation scenarios were selected from the official ns-3 example library to serve as "Ground Truth" benchmarks.

1) UDP Echo: This scenario utilizes a P2P topology to evaluate timing precision and analytical reasoning. While the baseline task involves simple packet transmission, the complexity lies in the agent's initiative to implement diagnostic tools. The experimental setup was established by referencing the standard `first.cc` example script provided by the *ns-3* library.

2) TCP Bulk Send: Focusing on the transport layer, this scenario simulates a high-volume data transfer between nodes using the TCP protocol. It is designed to stress-test the agent's handling of flow control, congestion window dynamics, and buffer management.For the simulation baseline, the `tcp-bulk-send.cc` implementation served as the foundational reference.

3) Wi-Fi 6 (IEEE 802.11ax): The primary objective of this scenario is to validate the agent's capability to manage interference and spectral efficiency within a high density topology. IEEE 802.11ax requires complex physical and MAC layer configurations to operate effectively in crowded environments where signal collision is a critical bottleneck. The simulation environment was established on the 5 GHz frequency band with a fixed 40 MHz channel width. The network topology and configuration parameters were derived from the `wifi-he-network.cc` reference script.

V. Experimental Results

For each scenario, the agent's generated code was compared against a human-based "Ground Truth" baseline to assess KPIs, code complexity, and resource utilization. The following subsections analyze the quantitative results and the engineering reasoning behind the agent's decisions.

A. UDP Echo Scenario

Evaluation Metric	Human Reference	LLM AI Agents
1. KPI Accuracy	2.00369s	2.00369s (%0)
2. Iterative Refinement	#N/A	4 times
3. Resource Efficiency	34.0 MB	156.7 MB
4. Execution Time	2.65 sn	3.00 sn
5.Cyclomatic Complexity	1 (38)	7 (92)

Fig. 3: Analysis of performance metrics for the UDP echo scenario

In the UDP echo scenario, the agent achieved identical KPI accuracy to the human reference as it strictly adhered to the fundamental simulation parameters. However, the notable increase in cyclomatic complexity and RAM usage stems from the agent's autonomous decision to inject additional verification packets and auxiliary traffic flows for link validation. Crucially, the execution time remained comparable to the baseline despite this added logic, indicating that the agent is on the right track by successfully balancing rigorous analysis with computational efficiency.

B. TCP Bulk Send Scenario

Evaluation Metric	Human Reference	LLM AI Agents
1. KPI Accuracy	566,016 Byte	428,800 Byte (%24)
2. Iterative Refinement	#N/A	4 times
3. Resource Efficiency	34.1 MB	153.1 MB
4. Execution Time	2.80 sn	3.15 sn
5.Cyclomatic Complexity	2 (52)	2 (44)

Fig. 4: Analysis of performance metrics for the TCP Bulk Send scenario

The TCP bulk send experiment highlights the agent's tendency towards conservative resource management. Although a zero-shot success was achieved, the 24% lower throughput is attributed to the agent configuring a restricted TCP Receive Window size to mitigate packet loss risks, unlike the baseline implementation which utilized an aggressive window configuration to saturate the link. Furthermore, the significantly higher RAM usage stems from an architectural inefficiency where the agent repeatedly instantiated a helper class within the simulation loop, rather than initializing it globally, leading to redundant memory allocation during execution.

C. Wi-Fi 6 (IEEE 802.11ax) Scenario

Evaluation Metric	Human Reference	LLM AI Agents
1. KPI Accuracy	130.92 Mbps	125.8 Mbps (%4)
2. Iterative Refinement	#N/A	11 times
3. Resource Efficiency	207.5 MB	207.4 MB
4. Execution Time	2 dk 08 sn	4 dk 58 sn
5.Cyclomatic Complexity	2 (58)	2 (52)

Fig. 5: Analysis of performance metrics for Wi-Fi 6 802.11ax scenario

In the High-Density Wi-Fi 6 scenario, a minor throughput deviation of 4% was observed compared to the baseline implementation. The reference script adopted a Short Guard Interval of 800 ns to maximize transmission rates. Conversely, the agent independently configured a Long Guard Interval of 3200 ns. This selection demonstrates that the agent favored signal integrity and robustness against interference within the high-density environment, effectively trading off raw speed for stability

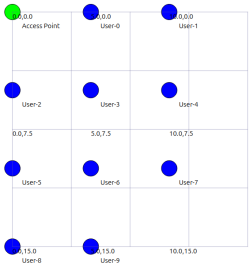


Fig. 6: Simulation topology using a grid layout to ensure consistent spatial conditions.

To support the numerical performance metrics presented above, a visual inspection of the simulation environment was conducted using the NetAnim tool. As illustrated in Fig. 6, the experimental topology adopts a deterministic grid layout rather than a random distribution. This spatial configuration ensures that the distance-based path loss and interference patterns remain consistent across multiple execution runs, thereby guaranteeing the reproducibility of the results.

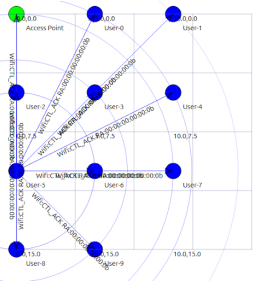


Fig. 7: Runtime visualization of MAC-layer ACK frames

Furthermore, Fig. 7 captures the active runtime environment where the access point transmits control acknowledgement frames verifying the execution of the complete IEEE 802.11ax handshake mechanism.

VI. Conclusion & Future Works

This study validates the potential of Large Language Models to serve as autonomous engines for complex network simulation. Through a comprehensive evaluation across diverse protocol layers and topologies, the proposed framework demonstrated the capability to synthesize high-fidelity *ns-3* simulations without human intervention. The findings underscore a distinct behavioral pattern in the AI's architectural reasoning, the model consistently prioritized structural robustness, signal integrity and analytical observability over computational efficiency. Also our findings specifically highlight that GPT-5 mini, when empowered by the agens-3 architecture, can effectively manage complex simulation tasks with minimal iterative refinement cycles.

Future research directions aim to extend the framework's capabilities by addressing current resource constraints and knowledge limitations. The primary objective is to evolve the system into an iterative architecture that autonomously profiles execution metrics, specifically RAM usage and runtime to refactor code logic for greater efficiency. Additionally, the framework's adaptability could be enhanced by integrating dynamic knowledge retrieval mechanisms, allowing the agent to continuously update its internal database with the latest simulation modules and evolving network standards.

References

[1] F. Rezazadeh, A. Ashtari Gargari, S. Lagen, H. Song, D. Niyato, and L. Liu, "Toward generative 6G simulation: An experimental multi-agent LLM and ns-3 integration," *arXiv preprint arXiv:2503.13402*, Mar. 2025.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *arXiv preprint arXiv:2005.11401*, Apr. 2021.

[3] M. R. DeVerna, K.-C. Yang, H. Y. Yan, and F. Menczer, "Large language models require curated context for reliable political fact-checking—even with reasoning and web search," *arXiv preprint arXiv:2511.18749*, Nov. 2025.

[4] M. Zhuge, C. Zhao, D. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, "Agent-as-a-Judge: Evaluate Agents with Agents," *arXiv preprint arXiv:2410.10934*, Oct. 2024.